Edge cases (a bad release that slips past the gate) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The core problem: `live_version.txt` currently stores exactly one piece of information — the version name — and nothing distinguishes "this got here through `deploy()` after passing the gate" from "someone typed a name into a text file." You need to store a second piece of information that only `deploy()` can produce correctly, and check it every time anything reads "what's live."

A plain hash isn't quite enough on its own — `hashlib.sha256(version_name.encode()).hexdigest()` is public knowledge; anyone who can read your `deploy_gate.py` source can recompute the same hash for any version name they like, and the tampered file would look valid again. What you actually need is `hmac`, Python's built-in module for exactly this: a hash that also depends on a **secret** value the editor doesn't have — so recomputing a valid-looking signature requires knowing that secret, not just knowing the algorithm.

The exact pieces:

- `import hmac, hashlib` — `hmac.new(secret.encode(), version_name.encode(), hashlib.sha256).hexdigest()` produces a signature that depends on both the version name *and* the secret.
- A secret string, stored somewhere `deploy_gate.py` reads but a casual file edit doesn't touch — for this exercise, a constant in the script or a separate `secret.txt` is fine; a real project would use an environment variable.
- `hmac.compare_digest(a, b)` — compares two signatures safely; use this instead of `a == b` (it avoids a subtle timing side-channel a plain `==` has, worth knowing even though it doesn't matter much for a local file).
- A JSON pointer file, `live_version.json`, holding `{"version_name": ..., "signature": ...}` instead of one bare text line.

Going further — the harder question:

Here's the deeper question Hint 1's Basic and Intermediate levels don't fully answer: **if the secret lives in the same repository as everything else, what actually stops someone who can edit `live_version.json` from also reading the secret and computing a valid signature themselves?** A signature check makes a *careless* hand-edit detectable — someone who forgets to recompute it, or doesn't know the check exists — but it doesn't stop a *deliberate* bypass by someone who reads the code first. That distinction matters: most real "gate bypass" incidents are the careless kind (someone under time pressure, editing the obvious file, not realizing there's a check) — but it's worth being honest about what a signature check does and doesn't defend against.

The stronger fix changes *where the truth lives*: instead of a separate pointer file being the source of truth for "what's live," make the version log itself (from `version_log`) the only source of truth — each version record gets a `passed_gate: true` field, written only by `deploy()` at the moment it passes, never by any other code path. `get_live_version()` doesn't read a pointer file at all; it reads the version log and returns the most recent entry with `passed_gate: true`. There's no separate file left to edit that would have any effect — editing the log's `passed_gate` field by hand is still *possible*, but now it means editing the exact same record-of-truth the whole system already trusts and audits, not a separate, easy-to-miss side door.

The extra pieces needed:

- Add `passed_gate: bool` (default `False`) to every version record when it's first saved.
- `deploy()` sets `passed_gate = True` on the target version's record (rewriting that one entry in the log) only inside its passing branch — never anywhere else in the codebase.
- `get_live_version()` scans the log for entries with `passed_gate == True`, sorted by `created_at`, and returns the most recent one — no separate pointer file needed at all.

Sketch `get_live_version()` built this way yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
SECRET = some string only this script knows

function save_pointer(version_name):
    signature = hmac(SECRET, version_name)
    write to live_version.json:
        {"version_name": version_name, "signature": signature}

function get_live_version() -> str | None:
    read live_version.json
    expected = hmac(SECRET, pointer["version_name"])
    if pointer["signature"] does not safely match expected:
```

```
        print a tamper warning, return None
    return pointer["version_name"]
```

```
# release_gate_practice.py — Edge cases section
import hmac
import hashlib
import json

SECRET = "replace-with-a-real-secret-not-committed-to-git"

def sign(version_name: str) -> str:
    # how: the signature depends on the name AND the secret
    key = SECRET.encode()
    message = version_name.encode()
    return hmac.new(key, message, hashlib.sha256).hexdigest()

def save_pointer(version_name: str) -> None:
    pointer = {"version_name": version_name, "signature": sign(version_name)}
    with open("live_version.json", "w") as f:
        json.dump(pointer, f)

def get_live_version() -> str | None:
    with open("live_version.json") as f:
        pointer = json.load(f)
    expected = sign(pointer["version_name"])
    if not hmac.compare_digest(pointer["signature"], expected):
        print("Warning: live pointer looks tampered with!")
        return None
    return pointer["version_name"]
```

Try it: `save_pointer("v2")`, confirm `get_live_version()` returns `"v2"`, then hand-edit the JSON file's `version_name` to `"v1"` and confirm `get_live_version()` now warns and returns `None`. Compare against the Solution.

Going further, as a plan:

```
every version record in versions.json gains: "passed_gate": False (by default)

function mark_passed(version_name):
    load versions
    find the matching record, set its passed_gate to True
    save versions back

function get_live_version() -> str | None:
    load versions
```

```
    keep only the ones where passed_gate is True
    if none: return None
    return the version_name of the one with the latest created_at
```

Here's almost the whole thing — fill in `mark_passed` yourself:

```
# release_gate_practice.py — Edge cases section
import json
from pathlib import Path

def load_versions(path: str = "versions.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_versions(versions: list[dict], path: str = "versions.json") -> None:
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

# your turn: write mark_passed(version_name, path="versions.json") — it
# should load the versions, find the one matching version_name, set its
# "passed_gate" key to True, and save the whole list back

def get_live_version(path: str = "versions.json") -> str | None:
    versions = load_versions(path)
    passing = []
    for v in versions:
        if v.get("passed_gate"):
            passing.append(v)

    if not passing:
        return None

    latest = passing[0]
    for v in passing:
        if v["created_at"] > latest["created_at"]:
            latest = v
    return latest["version_name"]
```

Fill in `mark_passed` yourself, then compare all 3 of your finished versions against the Solution.

Full solution: Show me the solution